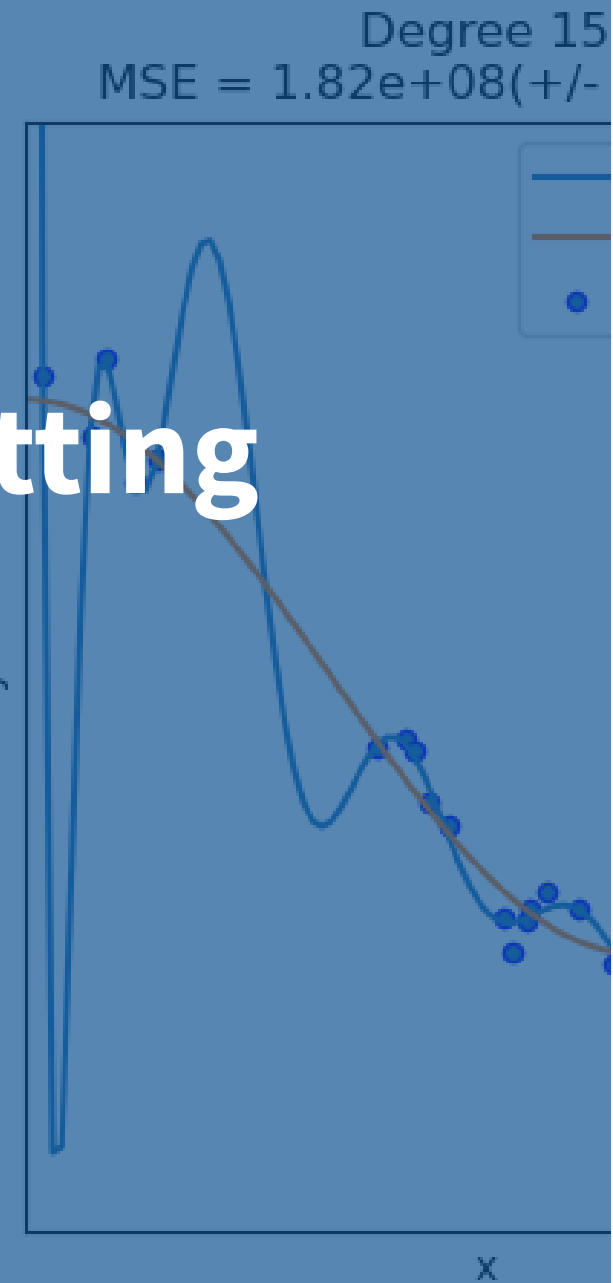
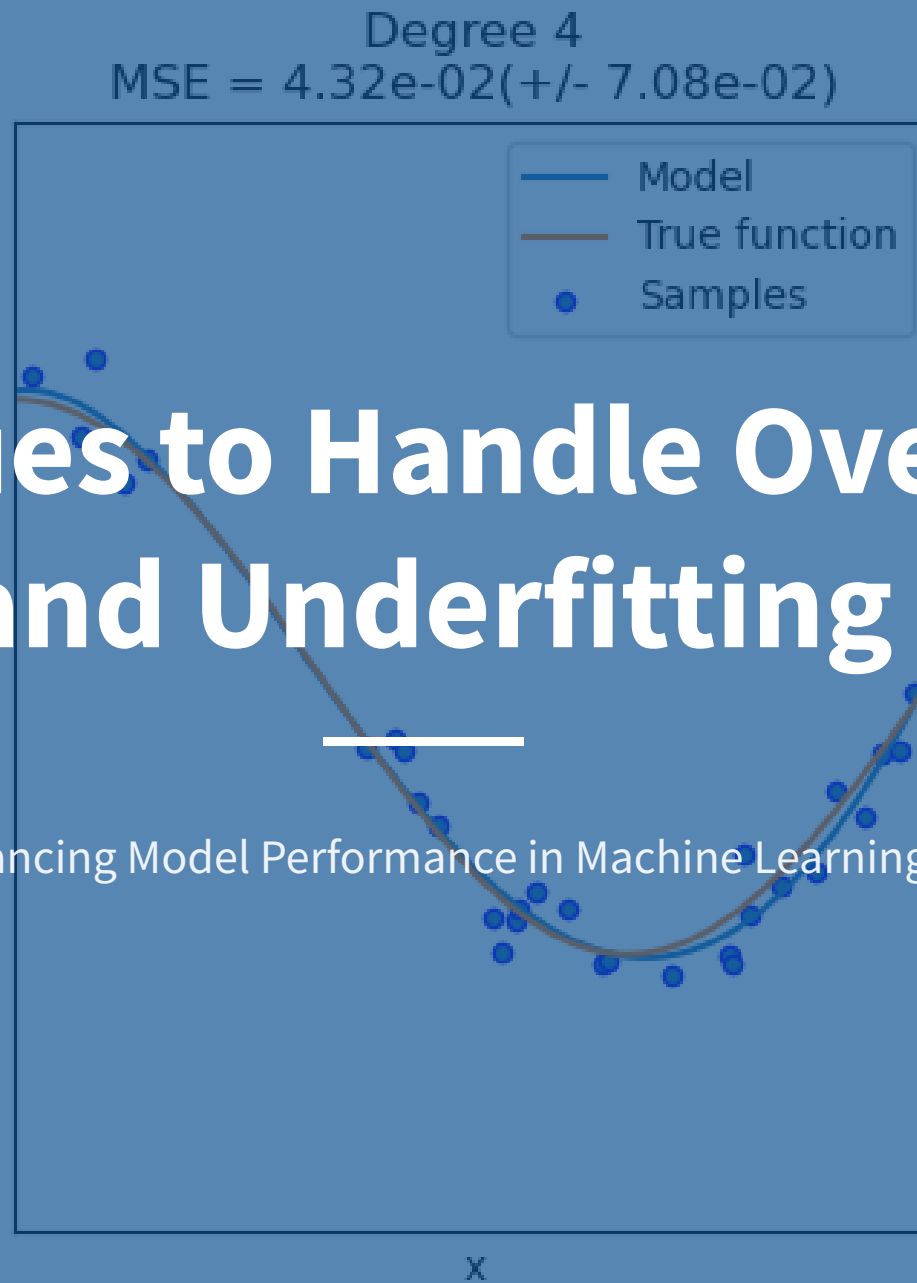




Techniques to Handle Overfitting and Underfitting

Balancing Model Performance in Machine Learning

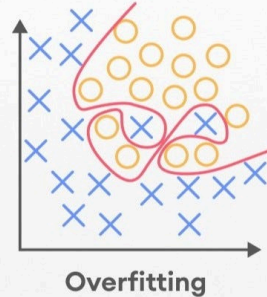
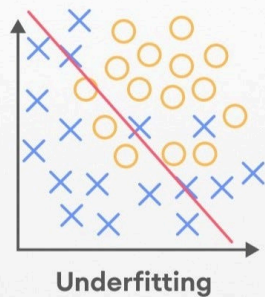
Understanding Overfitting and Underfitting

↗ Overfitting

Model learns training data too well, capturing noise and random fluctuations

Impact:

- ✓ Excellent on training data
- ✗ Poor generalization to new data

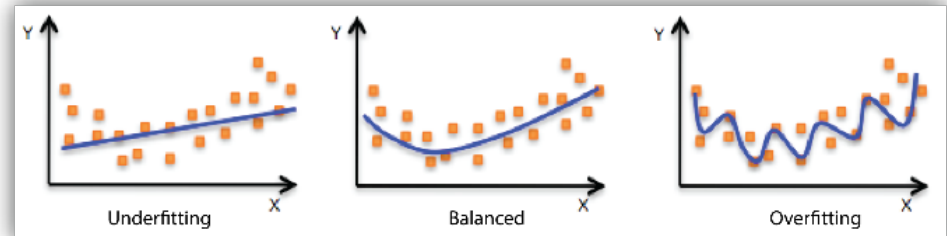


↘ Underfitting

Model is too simple to capture underlying patterns in the data

Impact:

- ✗ Poor performance on training data
- ✗ Poor performance on new data



The Bias-Variance Tradeoff

↘ Bias

Error from **oversimplified assumptions** in the learning algorithm

High bias → Underfitting

Model fails to capture underlying patterns

↗ Variance

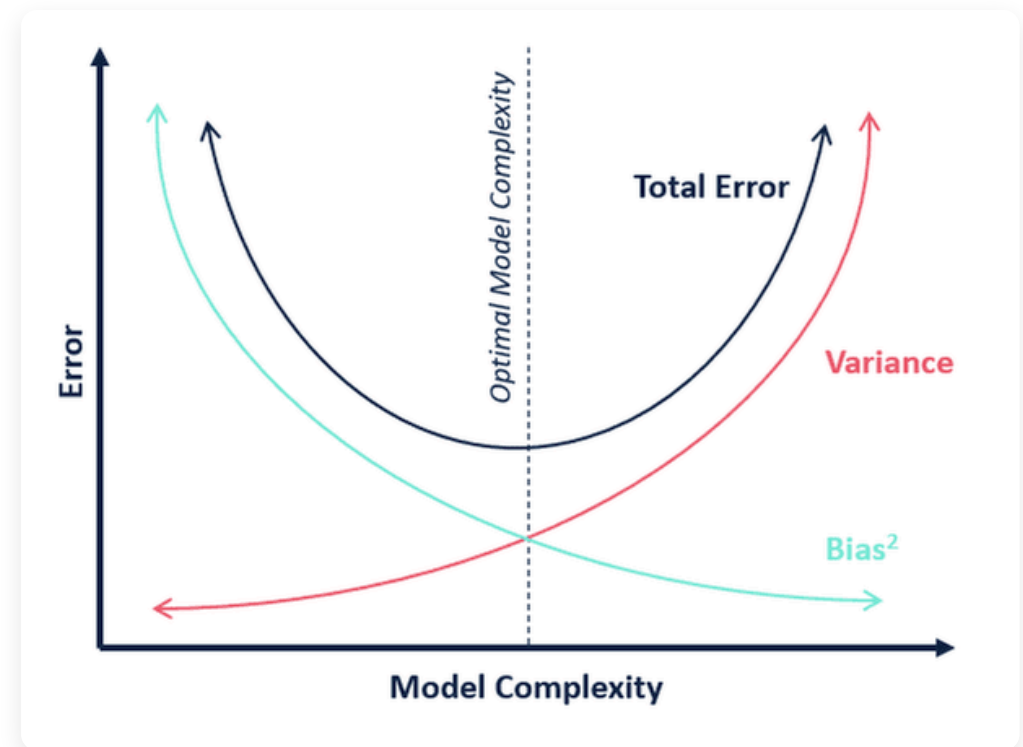
Error from **sensitivity to small fluctuations** in the training set

High variance → Overfitting

Model captures noise in training data

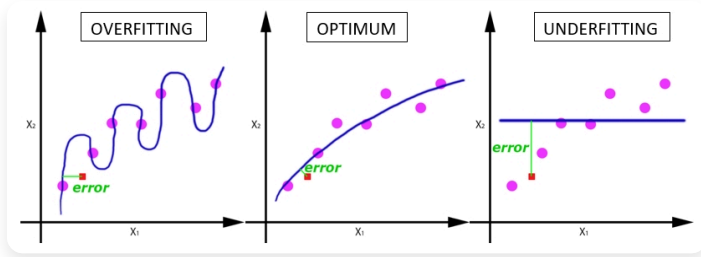
The Tradeoff

- ⚖ Decreasing bias increases variance and vice versa
- ✳ Goal: Find optimal balance between bias and variance
- 💡 Total Error = $\text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$



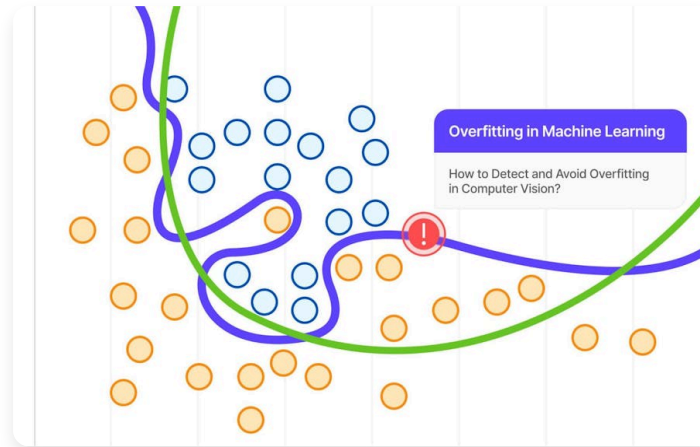
Visualizing Overfitting and Underfitting

Underfitting



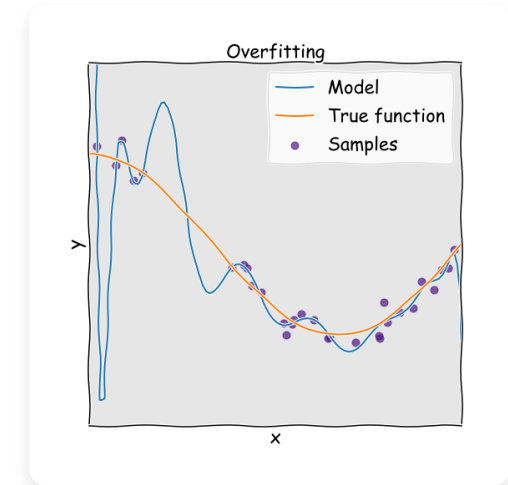
- ❗ High bias, low variance
- ❗ Poor performance on training data
- ❗ Model too simple for data complexity

Optimal Fitting



- ✅ Balanced bias and variance
- ✅ Good performance on training data
- ✅ Generalizes well to new data

Overfitting



- ⚠️ Low bias, high variance
- ⚠️ Excellent on training data
- ⚠️ Poor generalization to new data

The goal is to find the sweet spot between underfitting and overfitting

Causes of Overfitting



High Model Complexity

Model with **too many parameters** relative to training data size



Insufficient Training Data

Limited data prevents model from learning general patterns



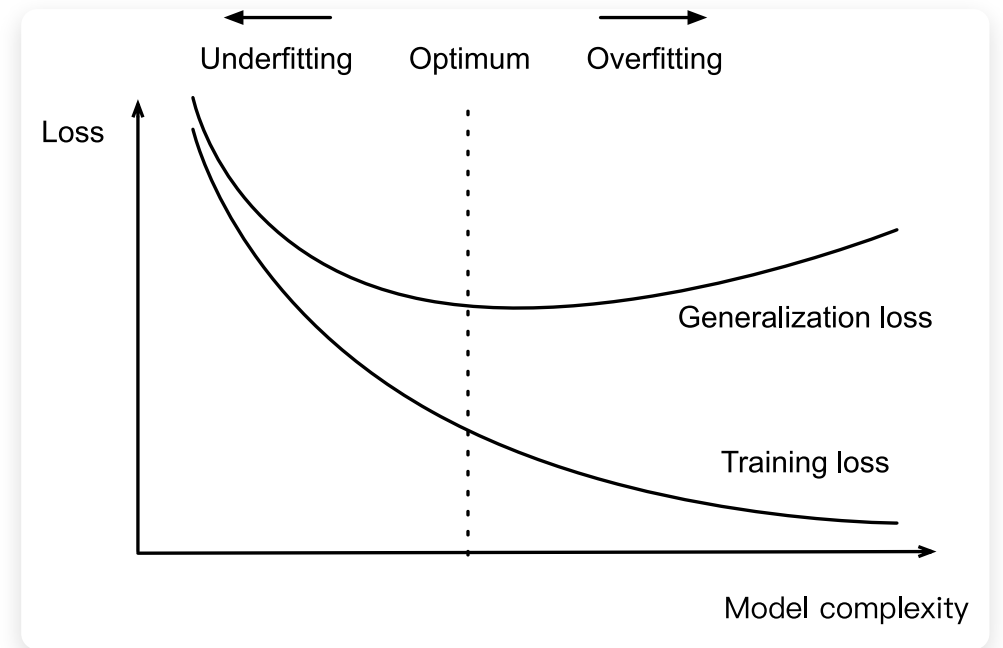
Noisy Data

Model learns **random fluctuations** and outliers instead of true patterns



Excessive Training

Training for **too many epochs** leads to memorization of training data



Causes of Underfitting



Low Model Complexity

Model is **too simple** to capture underlying patterns in data



Insufficient Features

Missing **important features** that explain the target variable



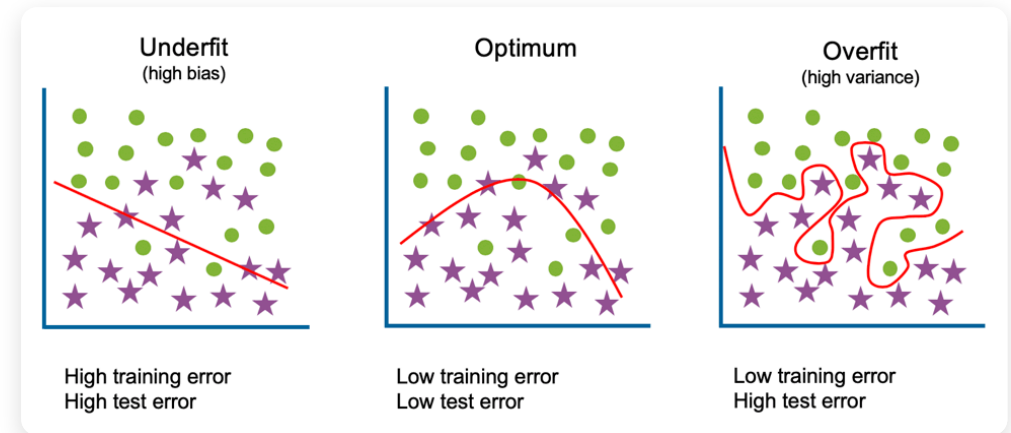
Excessive Regularization

Too strong penalties prevent model from learning patterns



Insufficient Training

Not enough epochs or iterations for model to learn



Detecting Overfitting and Underfitting



Performance Metrics Analysis

- ↗ High training accuracy, low test accuracy
- ↘ Low accuracy on both sets
- ↔ Similar performance on both sets



Learning Curves

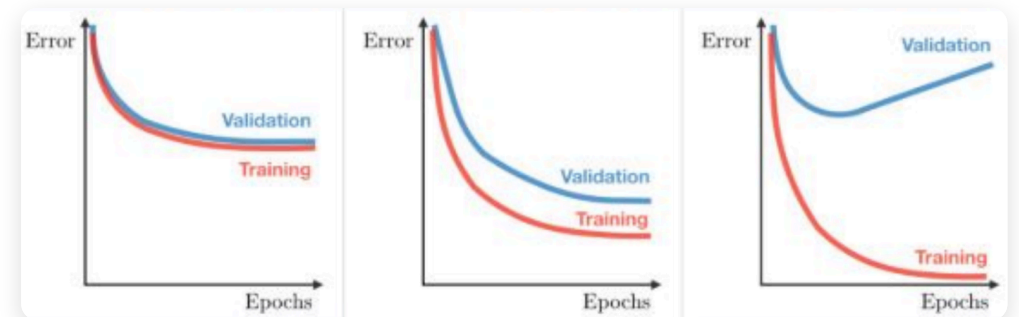
- ↗ Gap between training and validation curves
- ↑ Validation error increasing after optimum
- ↓ Both errors plateau at high level



Validation Techniques

- ↔ Cross-validation performance variance
- ✂ Performance consistency across folds
- ▲ Performance on different data subsets

Learning Curves Visualization



Techniques to Handle Overfitting

Overfitting occurs when models learn training data too well, capturing noise and random fluctuations. These techniques help improve generalization:



Cross-Validation

Split data into **multiple folds** to evaluate model performance on unseen data



Regularization

Add **penalty terms** (L1/L2) to constrain model complexity



Dropout

Randomly **disable neurons** during training to prevent co-adaptation



Early Stopping

Monitor **validation error** and stop training when it starts increasing



Data Augmentation

Artificially expand training data with transformations



Feature Selection

Choose only **relevant features** to reduce model complexity

Each technique addresses overfitting from a different angle. The best approach often combines multiple techniques.

Cross-Validation

K-Fold Cross-Validation

- Split dataset into **k equal folds**
- Train on **k-1 folds**, validate on remaining fold
- Repeat process **k times**, using each fold as validation once
- Average results to get **robust performance estimate**

Benefits for Overfitting Prevention

- More reliable evaluation
- Uses all data for training
- Better model selection
- Optimal hyperparameter tuning



Regularization (L1 and L2)

1 L1 Regularization

Adds **absolute value** of coefficients to loss function

- ✓ Creates **sparse models**
- ✓ Drives **some weights to exactly zero**
- ✓ Useful for **feature selection**

2 L2 Regularization

Adds **squared value** of coefficients to loss function

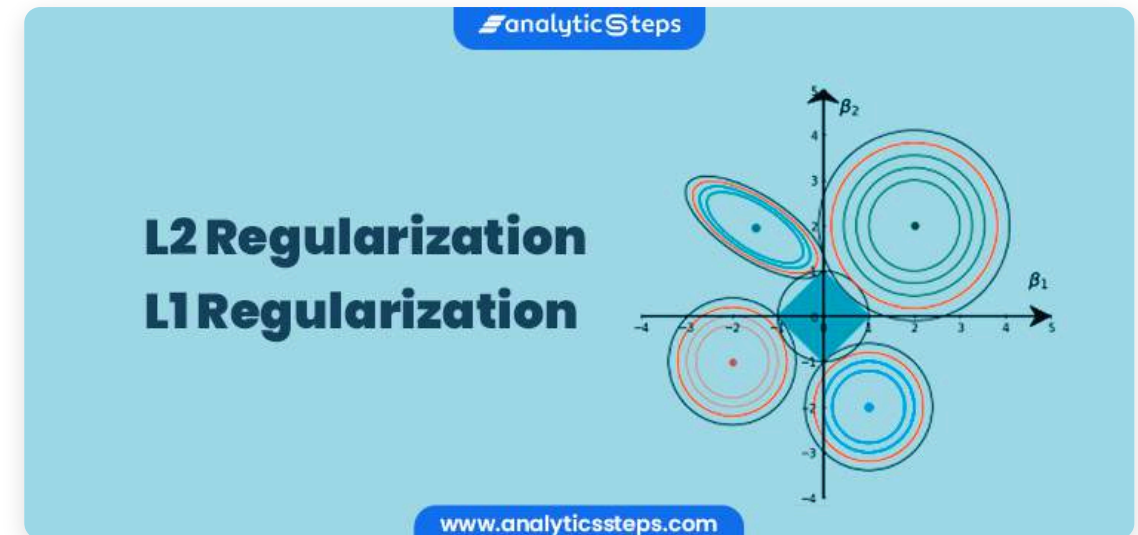
- ✓ Creates **dense models**
- ✓ **Shrinks weights** toward zero
- ✓ More **computationally stable**

Regularization Formula

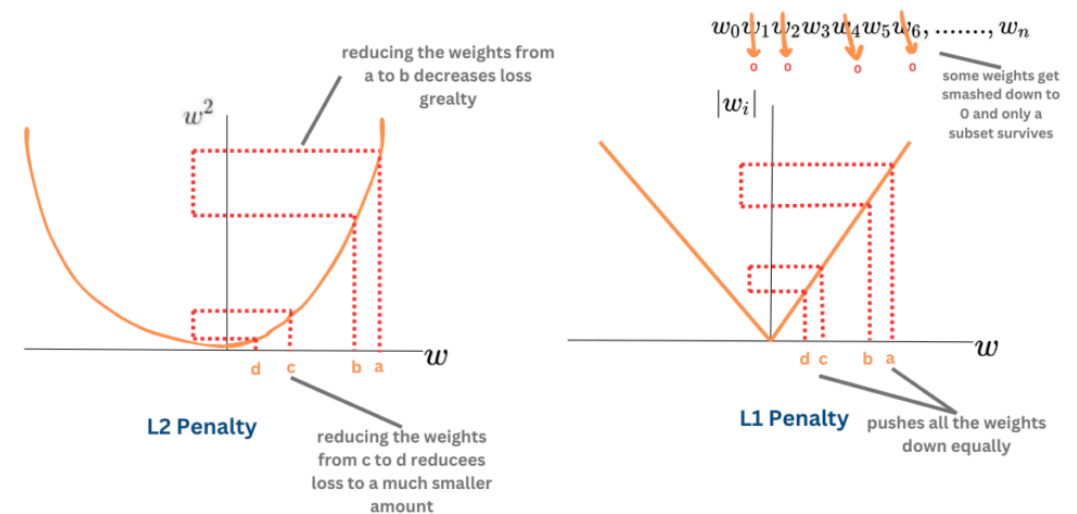
$$\text{Loss}_{\text{regularized}} = \text{Loss}_{\text{original}} + \lambda \times \text{Penalty}$$

Benefits for Overfitting Prevention

- ⚙ Controls model complexity
- 📉 Reduces variance
- ⚙ Prevents large coefficient values
- 🔄 Improves generalization



L1 and L2 Regularization Penalty



Dropout

✂ How Dropout Works

✂ Randomly deactivates neurons during training

% Typical dropout rates: 20-50% of neurons

↔ Forces network to learn **redundant representations**

↻ During inference, **all neurons active** with scaled weights

<> Implementation

- 1 Add dropout layers between hidden layers
- 2 Set appropriate dropout rate (0.2-0.5)
- 3 Only apply during training, not inference

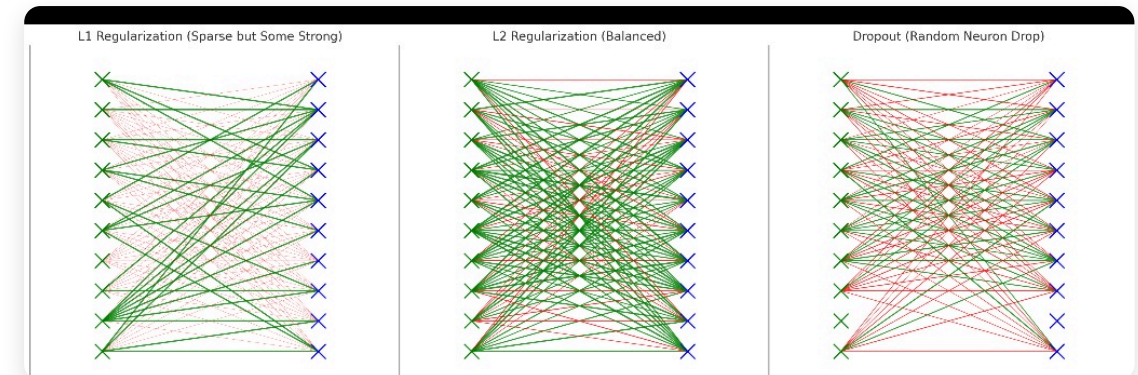
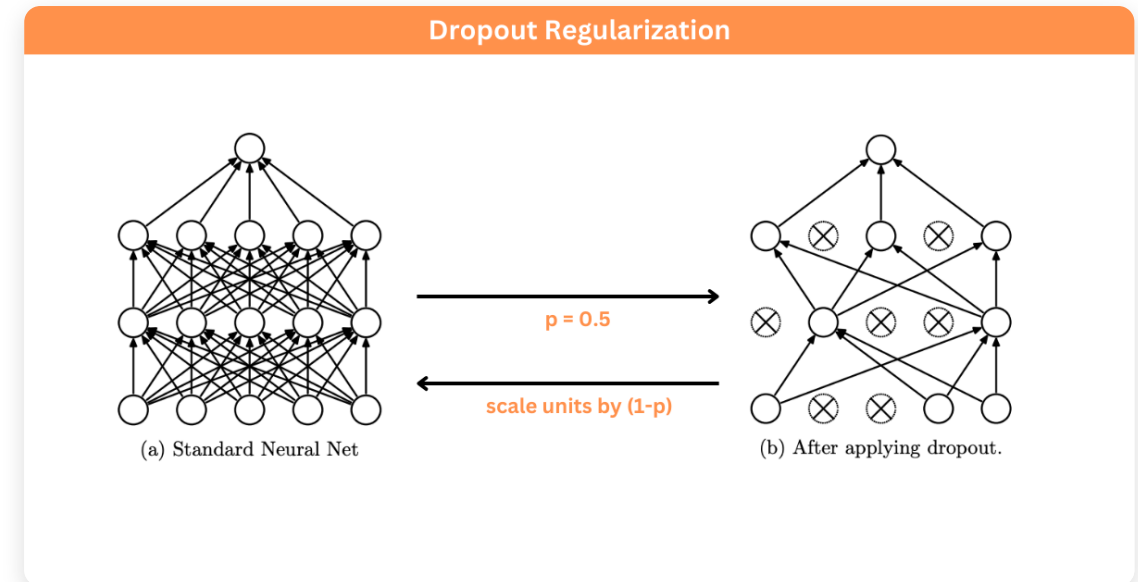
Benefits for Overfitting Prevention

👥 Reduces co-adaptation

🔗 Ensemble-like effect

✂ Improves generalization

📈 Faster training convergence



Early Stopping

How Early Stopping Works

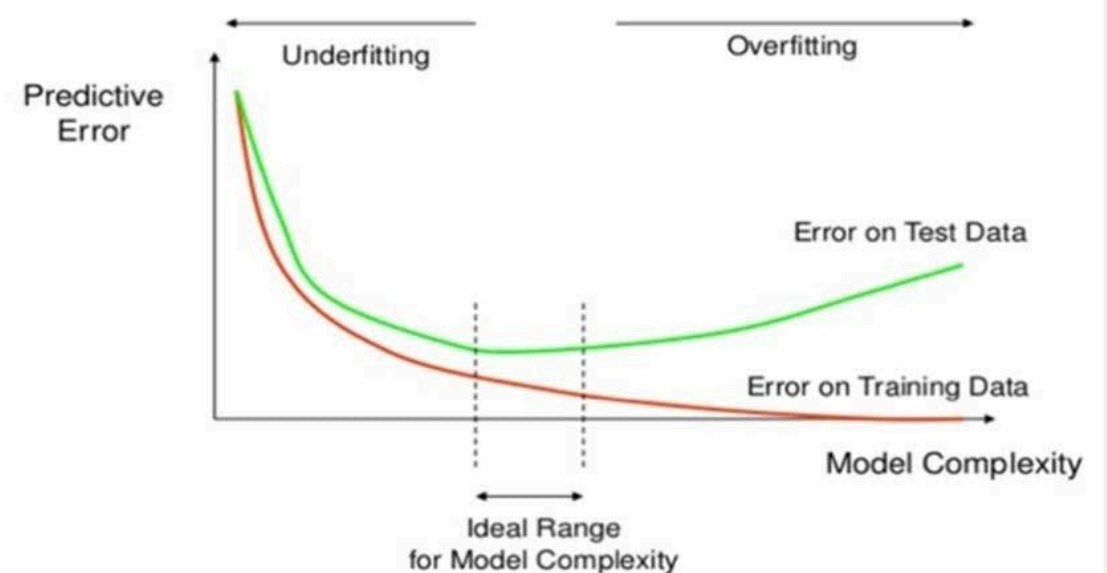
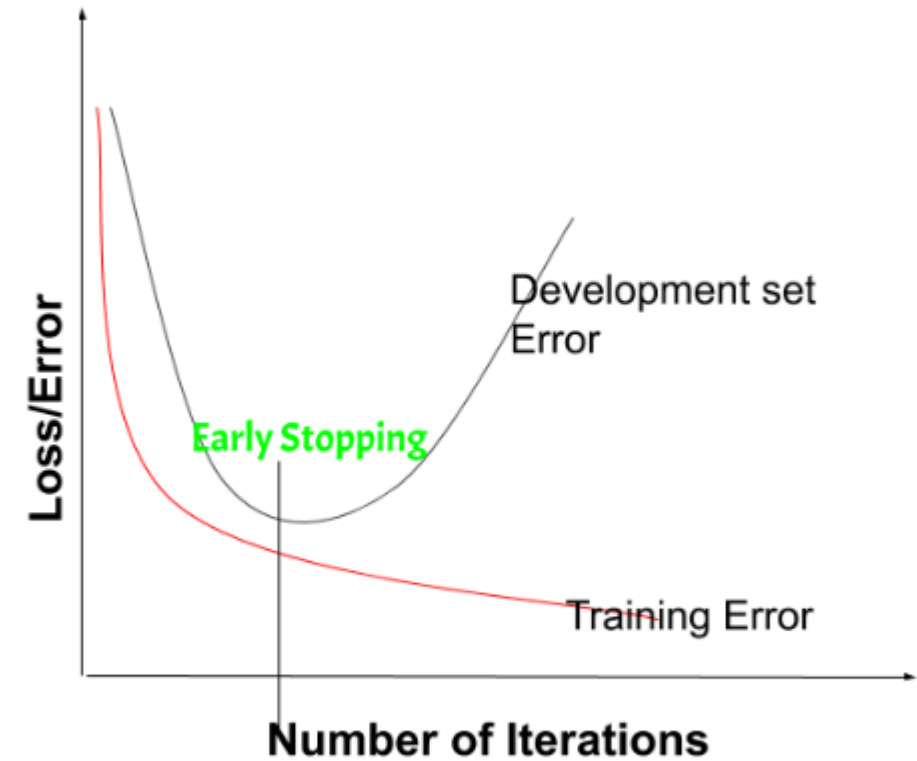
- Monitor validation loss during training
- Stop when validation loss starts increasing
- Save model weights from best performing epoch
- Can include patience parameter to avoid premature stopping

<> Implementation

- 1 Split data into training and validation sets
- 2 Track validation loss after each epoch
- 3 Set patience parameter (e.g., 5-10 epochs)
- 4 Stop training if no improvement after patience epochs

Benefits for Overfitting Prevention

- Prevents unnecessary training
- Optimal model selection
- Computational efficiency
- Better generalization



Data Augmentation

How Data Augmentation Works

 Artificially expand training dataset size

 Create **variations** of existing data samples

 Improves model **generalization** capabilities

 Flipping

 Rotation

 Cropping

 Scaling

 Brightness

 Contrast

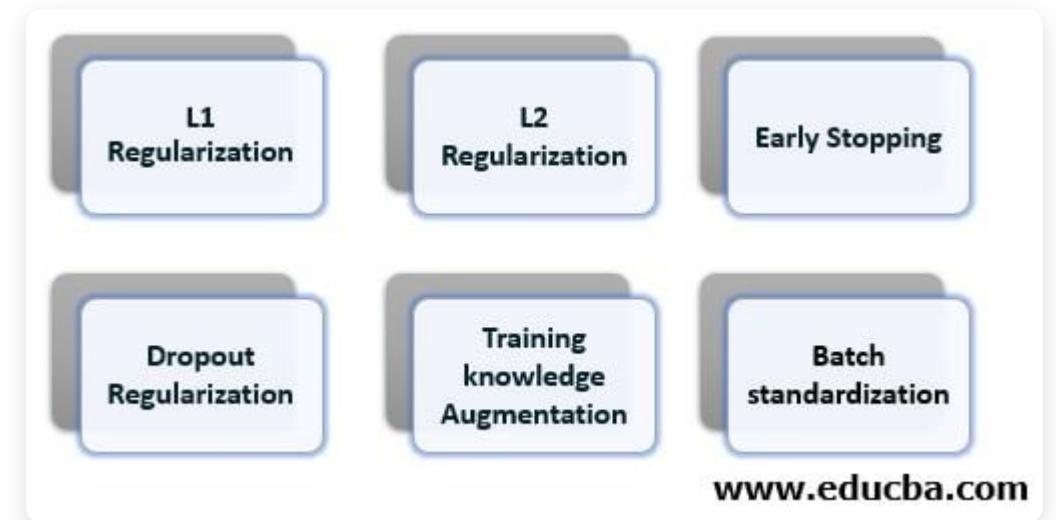
Benefits for Overfitting Prevention

 Increases effective data size

 Improves generalization

 Reduces need for more data

 Makes model robust to variations



Feature Selection

☰ How Feature Selection Works

☰ Reduces model complexity by selecting only relevant features

📊 Removes **irrelevant or redundant** features

🔄 Improves model **generalization** and interpretability



Filter Methods



Wrapper Methods



Embedded Methods



Statistical Tests

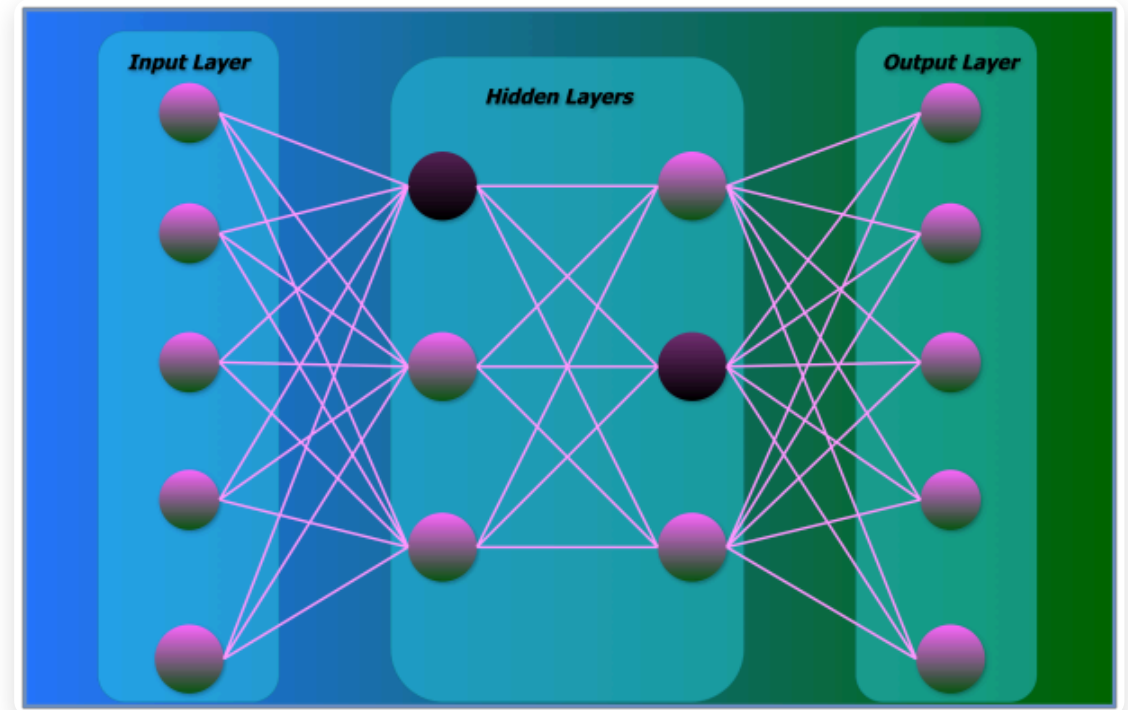
Benefits for Overfitting Prevention

🔄 Faster training

🔧 Better generalization

🧠 Improved interpretability

💾 Reduced memory usage



L1 Regularization

$$\text{Modified loss function} = \text{Loss function} + \lambda \sum_{i=1}^n |w_i|$$

L2 Regularization

$$\text{Modified loss function} = \text{Loss function} + \lambda \sum_{i=1}^n w_i^2$$

Techniques to Handle Underfitting

Underfitting occurs when models are too simple to capture underlying patterns in data. These techniques help improve model performance:



Increase Model Complexity

Use **more complex algorithms** with higher capacity



Feature Engineering

Add **new features** that better represent the data



Increase Training Duration

Train for **more epochs** or iterations



Reduce Regularization

Lower **penalty strength** to allow more flexibility

Unlike overfitting, underfitting requires increasing model capacity or improving data representation to capture true patterns.

Increasing Model Complexity

🧑 How to Increase Model Complexity

➕ Add **more parameters** to the model

📦 Increase **depth** in neural networks

Increase **width** by adding more neurons

⬆️ Choose **more complex algorithms**

Σ Higher-degree polynomials

🌳 More decision tree depth

🔗 More ensemble methods

🧠 More complex activation functions

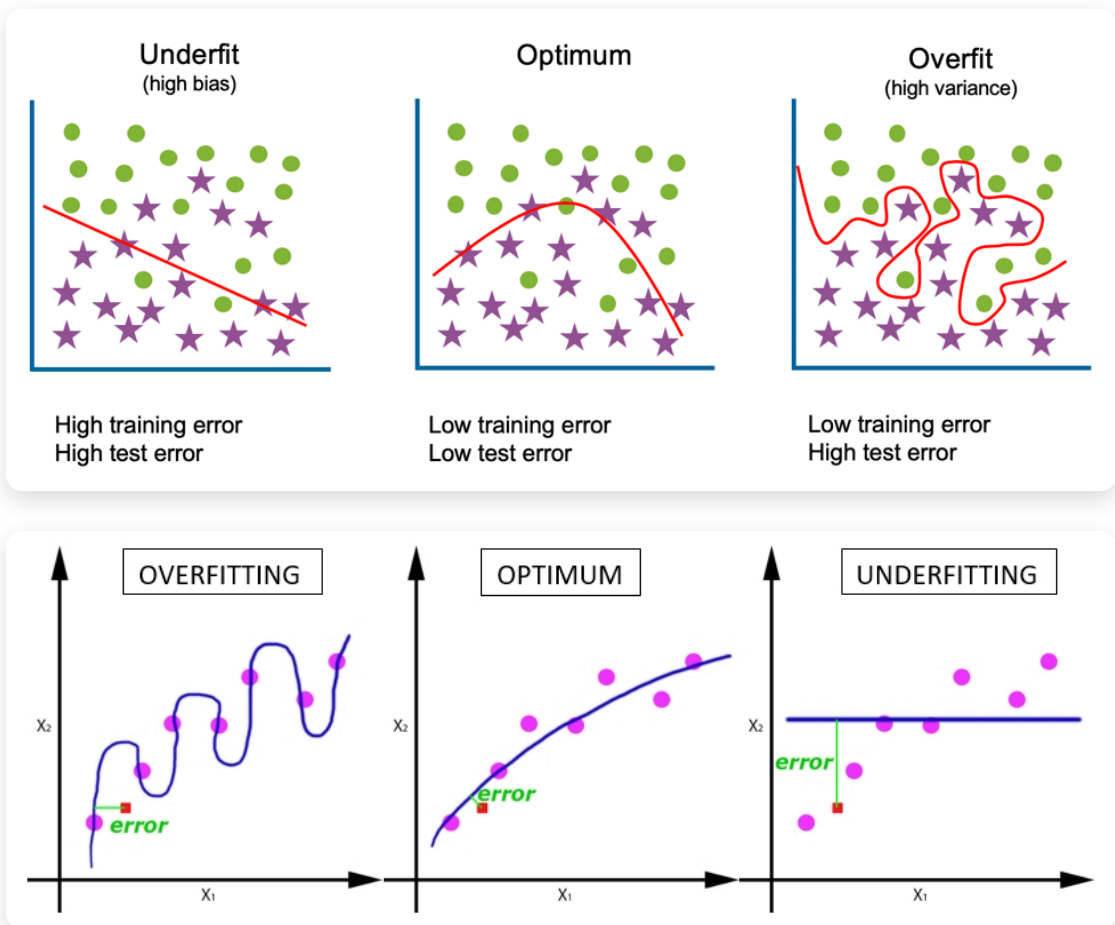
Benefits for Underfitting Prevention

↗️ Higher model capacity

🔗 Better pattern capture

↗️ Reduced bias

📊 Improved training performance



Feature Engineering

How Feature Engineering Works

+ Create **new features** from existing ones

↗ **Transform** features to better represent patterns

📊 Provide model with **more relevant information**

🔢 Mathematical Transformations

🏠 Categorical Encoding

👤 Feature Interactions

📈 Polynomial Features

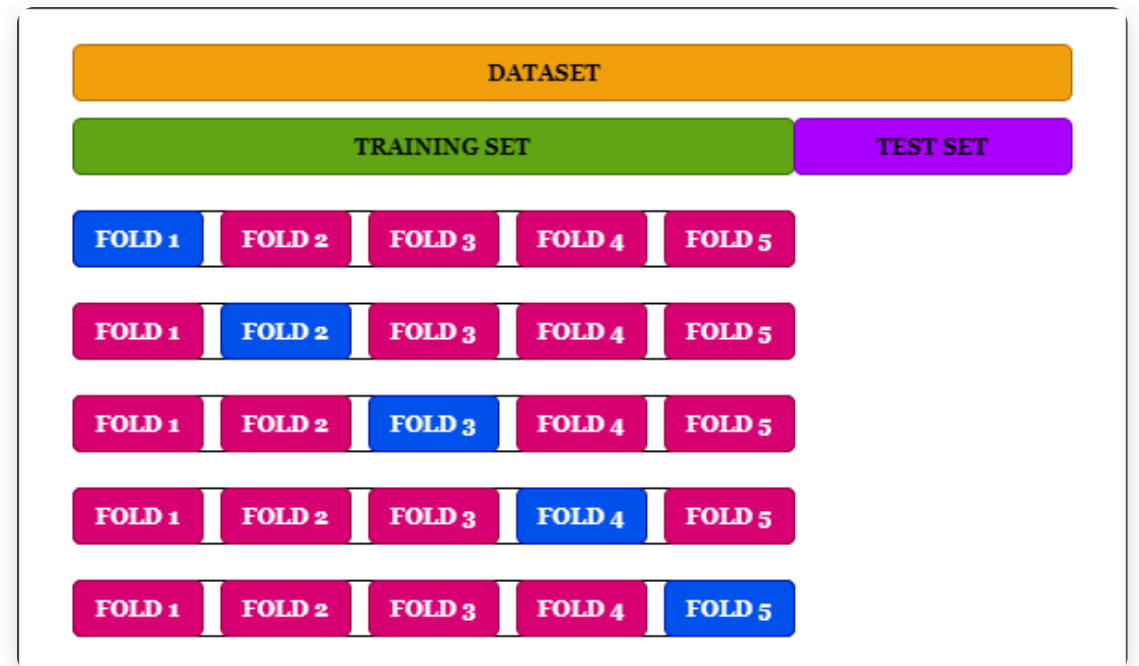
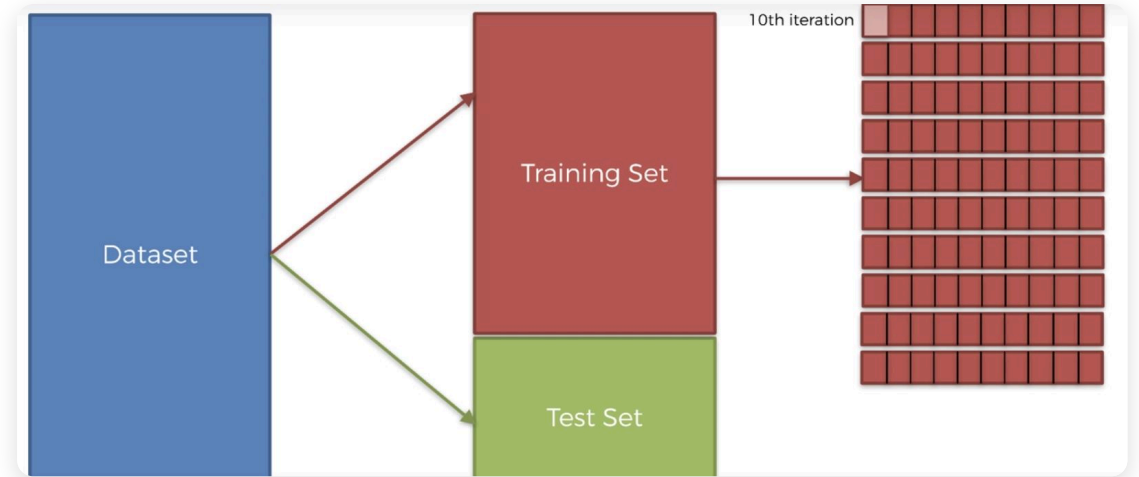
Benefits for Underfitting Prevention

🔧 Better pattern capture

📉 Reduced bias

🧠 Improved model understanding

⚡ Faster convergence



Increasing Training Duration

How More Training Helps

🔄 Train for **more epochs** to allow full learning

↘️ **Reduces training loss** by optimizing parameters

🎓 Model gets **more exposure** to data patterns

📈 Adjust learning rate

📅 Use learning schedules

🔄 Batch size optimization

📊 Gradient accumulation

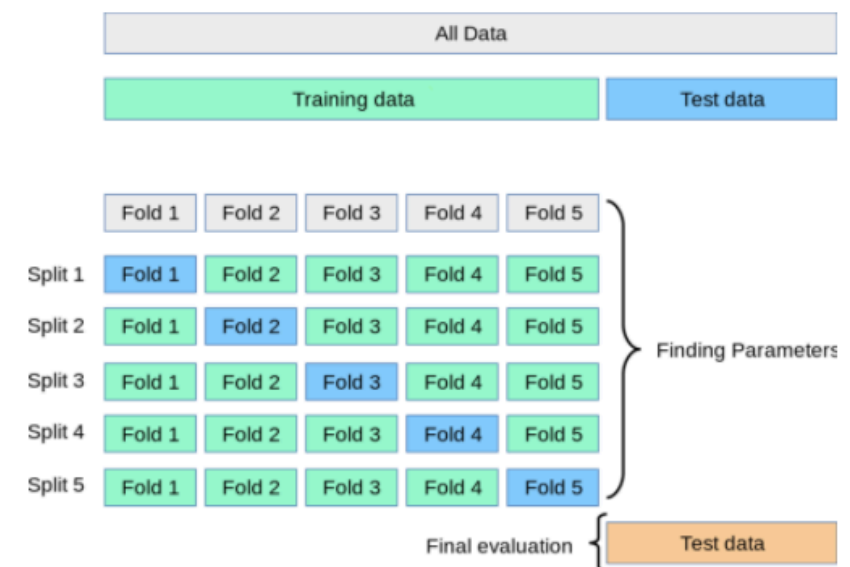
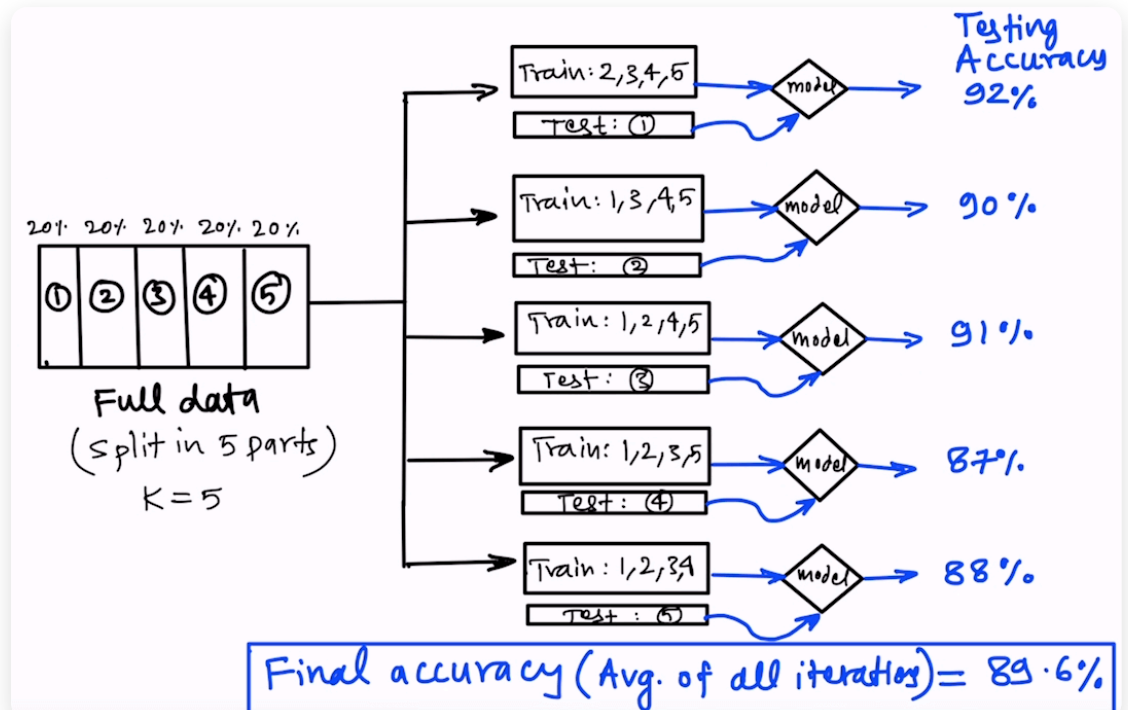
Benefits for Underfitting Prevention

↘️ Lower training error

🔍 Better pattern capture

↘️ Reduced bias

🔗 Improved convergence



Reducing Regularization

☰ How Reducing Regularization Helps

➡ Lower **penalty strength** allows more flexibility

↗ **Less constraint** on model parameters

✂ Model can capture **more complex patterns**

☰ Decrease λ (lambda) parameter

↗ Reduce dropout rate

1 Lower L1 regularization

2 Lower L2 regularization

Benefits for Underfitting Prevention

↗ Reduced bias

✂ Better pattern capture

⬆ Improved training performance

↘ Lower training error

L1 Regularization

$$\text{Modified loss function} = \text{Loss function} + \lambda \sum_{i=1}^n |W_i|$$

L2 Regularization

$$\text{Modified loss function} = \text{Loss function} + \lambda \sum_{i=1}^n W_i^2$$

L1
Regularization

L2
Regularization

Early Stopping

Dropout
Regularization

Training
knowledge
Augmentation

Batch
standardization

www.educba.com

Summary and Best Practices

↗ Overfitting Techniques



Cross-Validation



Regularization (L1/L2)



Dropout



Early Stopping



Data Augmentation



Feature Selection

↘ Underfitting Techniques



Increase Model Complexity



Feature Engineering



Increase Training Duration



Reduce Regularization



Best Practices



Find **balance** between bias and variance



Compare multiple techniques



Use **learning curves** to diagnose issues



Start **simple**, then add complexity

The right technique depends on your specific problem, data, and model architecture